Bayesian multimodeling: Approximate inference

MIPT

2026

Recap: Bayesian Inference and Model Selection

Parameter inference:

$$p(\mathbf{w}|\mathcal{D}, f) \propto p(\mathcal{D}|\mathbf{w}, f)p(\mathbf{w}|f)$$

Bayesian model selection (Evidence):

$$p(\mathcal{D}|f) = \int p(\mathcal{D}|\mathbf{w}, f)p(\mathbf{w}|f)\, d\mathbf{w}$$

Information-theoretic model selection (MDL):

$$\mathrm{MDL}(f, \mathcal{D}) = L(f) + L(\mathcal{D}|f)$$

Both Evidence and MDL balance goodness of fit and model complexity.

Variational calculus

Variational calculus problem is to find maxima and minima of functionals: mappings from a set of functions to the real numbers.

Example

Find a PDF p that gives maximum of entropy $H = -\int_w \log p(w) p(w) dw$.

- p — PDF to find (subject to constraints, e.g., fixed variance)
- H — functional

If a function is set from a predefined set of functions, we can consider the variational calculus problem as an approximation problem.

Recap: Evidence for Linear Regression

Model:

$$\boldsymbol{y}|\boldsymbol{X},\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{X}\boldsymbol{w}, \beta^{-1}\boldsymbol{I}), \qquad \boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{A}^{-1}).$$

Define

$$\boldsymbol{H} = \boldsymbol{A} + \beta \boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}, \qquad \hat{\boldsymbol{w}} = \beta \boldsymbol{H}^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{y}.$$

The Evidence is obtained by integrating out the parameters:

$$p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{A},\beta) = \int p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{w},\beta)p(\boldsymbol{w}|\boldsymbol{A})\, d\boldsymbol{w}.$$

For the Gaussian model, the integral has a closed-form solution:

$$\boxed{p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{A},\beta) = \frac{\beta^{m/2}|\boldsymbol{A}|^{1/2}}{(2\pi)^{m/2}|\boldsymbol{H}|^{1/2}} \exp[-S(\hat{\boldsymbol{w}})]}$$

where

$$S(\hat{\boldsymbol{w}}) = \underbrace{\frac{\beta}{2}\|\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}}\|^2}_{\text{data fit}} + \underbrace{\frac{1}{2}\hat{\boldsymbol{w}}^{\mathsf{T}}\boldsymbol{A}\hat{\boldsymbol{w}}}_{\text{prior penalty}}.$$

Laplace Approximation of the Evidence

Recall that the Evidence is an integral over the parameter space:

$$p(\mathfrak{D}|\boldsymbol{h}) = C \int \exp\bigl(-S(\boldsymbol{w})\bigr)\, d\boldsymbol{w},$$

where C is the normalization constant.

1. Approximate $S(\boldsymbol{w})$ around its minimum $\hat{\boldsymbol{w}}$ using a second-order Taylor expansion:

$$S(\boldsymbol{w}) \approx S(\hat{\boldsymbol{w}}) + \frac{1}{2}\Delta\boldsymbol{w}^{\mathsf{T}}\boldsymbol{H}\Delta\boldsymbol{w},$$

where $\Delta\boldsymbol{w} = \boldsymbol{w} - \hat{\boldsymbol{w}}$ and $\boldsymbol{H} = \nabla^2 S(\hat{\boldsymbol{w}})$.

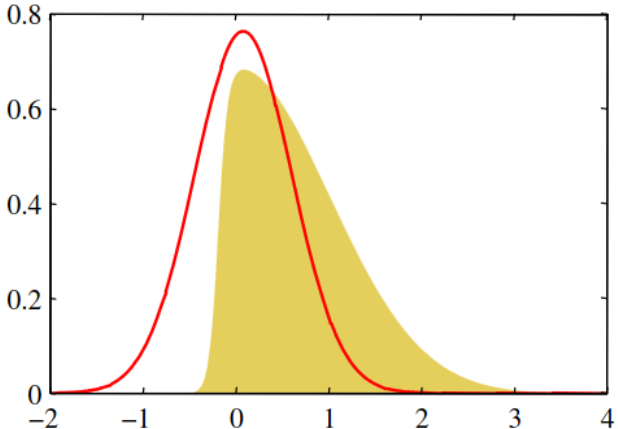
2. Evaluate the resulting Gaussian integral

$$p(\mathfrak{D}|\boldsymbol{h}) \approx C\, e^{-S(\hat{\boldsymbol{w}})} \frac{(2\pi)^{n/2}}{|\boldsymbol{H}|^{1/2}}.$$

Key idea: approximate the integrand locally by a Gaussian.

This makes Evidence estimation tractable for many nonlinear models.

Laplace approximation: example



Bishop, 2006

Laplace approximation: drawbacks

- Gaussian approximation around a single mode
  - Cannot capture multimodality
  - May be inaccurate for strongly non-Gaussian posteriors
- Computational cost of the Hessian
  - Storing and inverting a dense Hessian is expensive
  - Diagonal approximations are cheaper but ignore parameter correlations

Scalable Laplace approximation for neural networks

Ritter et al., 2018

- Approximate posterior curvature layer by layer, assuming a block-diagonal structure across layers.
- Within each layer, approximate the curvature matrix using a Kronecker-factored structure:

$$\mathbf{H}_l \approx \left(\mathbf{f}_l(\mathbf{h}_l)\mathbf{f}_l(\mathbf{h}_l)^\mathsf{T}\right) \otimes \mathbf{H}(\mathbf{h}_l).$$

- Exploit the Kronecker structure to simplify matrix inversion:

$$\mathbf{H}_l^{-1} \approx \left(\mathbf{f}_l\mathbf{f}_l^\mathsf{T}\right)^{-1} \otimes \mathbf{H}(\mathbf{h}_l)^{-1}.$$

For details, see the talk by Aleksandr Kolesov, 2021.

Local variational optimization: idea

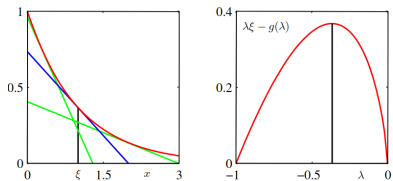
Consider approximating $f(x) = e^{-x}$ by a linear function.
By convexity, the tangent at x_0 is a global lower bound:

$$y(x) = f(x_0) + f'(x_0)(x - x_0).$$

Let $\lambda = f'(x_0) = -e^{-x_0} < 0$. Then

$$y(x) = \lambda x - \lambda + \lambda \log(-\lambda).$$

Where does variational optimization enter?



Bishop (2006): the red curve is the function; the blue line is its tangent.

Local variational optimization: idea

Consider approximating $f(x) = e^{-x}$ by a linear function.

By convexity, the tangent at x_0 is a global lower bound:

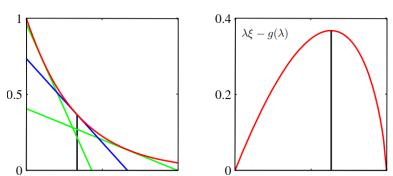
$$y(x) = f(x_0) + f'(x_0)(x - x_0).$$

Let $\lambda = f'(x_0) = -e^{-x_0} < 0$. Then

$$y(x) = \lambda x - \lambda + \lambda \log(-\lambda).$$

We seek the tightest lower bound:

$$\max_{\lambda<0} \left[\lambda x - \lambda + \lambda \log(-\lambda)\right].$$



Bishop (2006): the red curve is the function; the blue line is its tangent.

Local variational optimization and Evidence

We can construct a similar bound for the sigmoid function:

$$\log \sigma(x) = \frac{x}{2} - \log\left(e^{x/2} + e^{-x/2}\right).$$

The Jaakkola–Jordan bound is quadratic in x:

$$\sigma(x) \geq \sigma(\xi) \exp\left[\frac{x-\xi}{2} - \lambda(\xi)(x^2 - \xi^2)\right],$$

where

$$\lambda(\xi) = \frac{\tanh(\xi/2)}{4\xi}.$$

The variational parameter ξ controls the tightness of the bound.

- The log-likelihood is replaced by a quadratic lower bound.
- With a Gaussian prior, the resulting approximate posterior remains Gaussian.
- This makes approximate inference and Evidence estimation tractable.

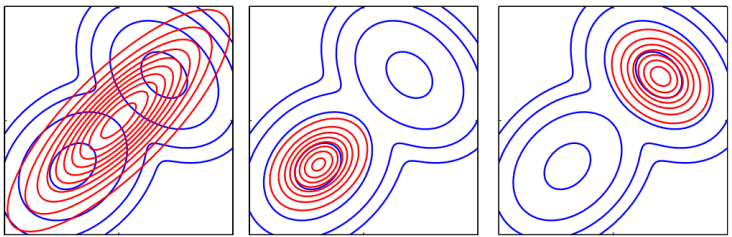
Evidence lower bound, ELBO

Evidence lower bound is a method of approximation of intractable distribution $p(\boldsymbol{w}|\mathfrak{D}, \boldsymbol{h})$ with a distribution $q(\boldsymbol{w}) \in \mathfrak{Q}$.

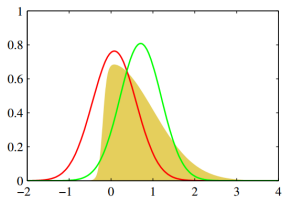
Evidence lower bound estimation often reduces to optimization problem

$$\log\ p(\mathfrak{D}|\boldsymbol{h}) = \mathsf{E}_{q(\boldsymbol{w})} \log\ p(\mathfrak{D}|\boldsymbol{w}) - \mathsf{KL}(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})) + \mathsf{KL}(q(\boldsymbol{w})||p(\boldsymbol{w}|\mathfrak{D})) \geq$$

$$\geq \mathsf{E}_{q(\boldsymbol{w})} \log\ p(\mathfrak{D}|\boldsymbol{w}) - \mathsf{KL}(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})).$$



Variational inference vs. expectation propagation (Bishop)



Laplace Approximation vs Variational inference

ELBO estimation

ELBO maximization

$$\int_{\boldsymbol{w}} q(\boldsymbol{w}) \log \frac{p(\boldsymbol{y}, \boldsymbol{w}|\boldsymbol{X}, \boldsymbol{h})}{q(\boldsymbol{w})} d\boldsymbol{w}$$

is equivalent to KL-divergence minimization between $q(\boldsymbol{w}) \in \mathfrak{Q}$ and posterior distribution $p(\boldsymbol{w}|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{h})$:

$$\hat{q} = \arg\max_{q \in \mathfrak{Q}} \int_{\boldsymbol{w}} q(\boldsymbol{w}) \log \frac{p(\boldsymbol{y}, \boldsymbol{w}|\boldsymbol{X}, \boldsymbol{h})}{q(\boldsymbol{w})} d\boldsymbol{w} \Leftrightarrow$$

$$\hat{q} = \arg\min_{q \in \mathfrak{Q}} \mathsf{D}_{\mathsf{KL}}\big(q(\boldsymbol{w}) || p(\boldsymbol{w}|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{h})\big),$$

$$\mathsf{D}_{\mathsf{KL}}\big(q(\boldsymbol{w}) || p(\boldsymbol{w}|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{h})\big) = \int_{\boldsymbol{w}} q(\boldsymbol{w}) \log \left(\frac{q(\boldsymbol{w})}{p(\boldsymbol{w}|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{h})} \right) d\boldsymbol{w}.$$

ELBO: normal distribution

"Common" loss function:

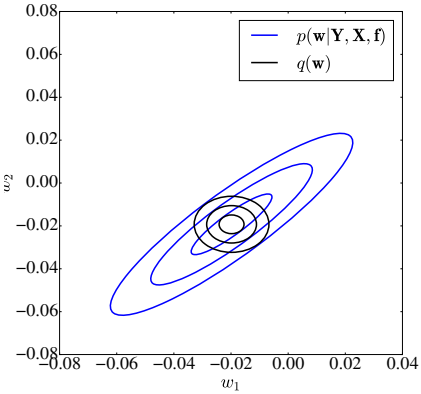
$$L = \sum_{\boldsymbol{x},\boldsymbol{y}\in\mathfrak{D}} -\log p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2.$$

Variational inference with $(p(\boldsymbol{w}|\boldsymbol{h}) \sim \mathcal{N}(\mathbf{0},\mathbf{1}))$:

$$L = -\sum_{\boldsymbol{x},\boldsymbol{y}} \log\, p(\boldsymbol{y}|\boldsymbol{x},\hat{\boldsymbol{w}}) +$$

$$+\frac{1}{2}\left(\mathrm{tr}(\boldsymbol{A}_q^{-1}) + \boldsymbol{\mu}_q^{\mathsf{T}}\boldsymbol{\mu}_q - \ln\,|\boldsymbol{A}_q^{-1}|\right).$$

Poor approximation example q



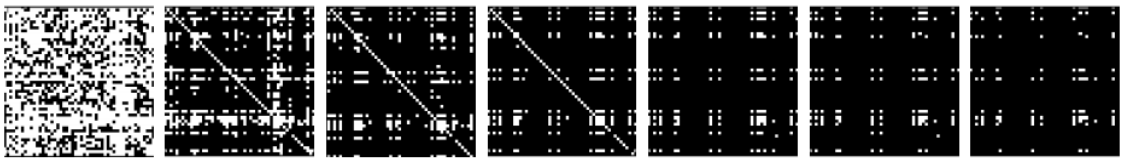
Graves, 2011

Prior: $p(\boldsymbol{w}|\sigma) \sim \mathcal{N}(\boldsymbol{\mu}, \sigma\boldsymbol{I})$.
Variational distribution: $q(\boldsymbol{w}) \sim \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\boldsymbol{I})$.
Greedy hyperparameter optimization:

$$\boldsymbol{\mu} = \hat{\mathsf{E}}\boldsymbol{w}, \quad \sigma = \hat{\mathsf{D}}\boldsymbol{w}.$$

Parameter pruning w_i using relative PDF:

$$\lambda = \frac{q(\mathbf{0})}{q(\boldsymbol{\mu}_{i,q})} = \exp(-\frac{\mu_i^2}{2\sigma_i^2}).$$



ELBO: normal distribution

Let $q \sim \mathcal{N}(\boldsymbol{\mu}_q, \boldsymbol{A}_q^{-1})$.
Then ELBO equals to:

$$\int_{\boldsymbol{w}} q(\boldsymbol{w}) \log p(\boldsymbol{Y}|\boldsymbol{X}, \boldsymbol{w}, \boldsymbol{h}) d\boldsymbol{w} - D_{\mathsf{KL}}\big(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})\big) \simeq$$

$$\sum_{i=1}^{m} \log p(\boldsymbol{y}_i|\boldsymbol{x}_i, \hat{\boldsymbol{w}}) - D_{\mathsf{KL}}\big(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})\big) \to \max_{\boldsymbol{A}_q^{-1}, \boldsymbol{\mu}_q}, \quad \hat{\boldsymbol{w}} \sim q.$$

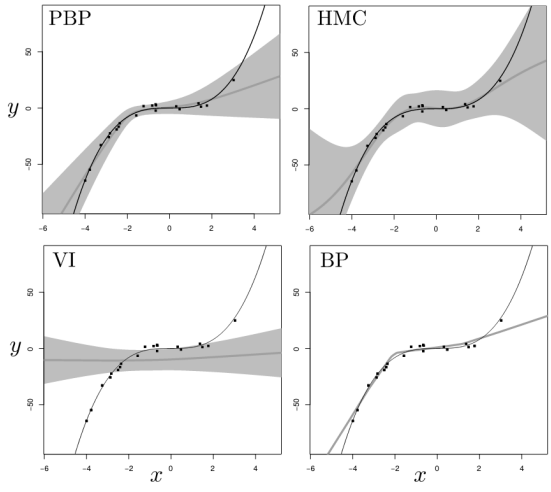
If prior $p(\boldsymbol{w}|\boldsymbol{h})$ is normal:

$$p(\boldsymbol{w}|\boldsymbol{h}) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{A}^{-1}),$$

KL-divergence $D_{\mathsf{KL}}\big(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})$ is computed analytically:

$$D_{\mathsf{KL}}\big(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})\big) = \frac{1}{2}\big(\mathsf{tr}(\boldsymbol{A}\boldsymbol{A}_q^{-1}) + (\boldsymbol{\mu} - \boldsymbol{\mu}_q)^{\mathsf{T}}\boldsymbol{A}(\boldsymbol{\mu} - \boldsymbol{\mu}_q) - n + \ln |\boldsymbol{A}^{-1}| - \ln |\boldsymbol{A}_q^{-1}|\big).$$

VI vs MC



What's better: ELBO or Laplace approximation?

(or MCMC)?

ELBO usage

ELBO: when to use?

- Evidence estimation;
- Latent distribution estimation (topic modeling, dimension reduction).

Why ELBO?

- reduces the problem of ELBO estimation to optimization;
- scales easily (compare with Laplace approximation;
- easy to use in comparison to MC-based methods.

ELBO can give a very biased evidence estimation.

Evidence estimation

We want to compute an expectation:

$$\mathbb{E}_{p(\mathbf{w})}[f(\mathbf{w})] = \int f(\mathbf{w})p(\mathbf{w})\, d\mathbf{w}.$$

- **Laplace approximation**
  - Gaussian approximation around a mode
  - Scales poorly
- **Variational inference**
  - Better scaling
  - Flexible choice of approximation family
  - ELBO provides a lower bound on log Evidence
- **Monte Carlo**
  - Estimates expectations using samples
  - Flexible, with accuracy depending on sampling quality
  - Can be computationally expensive

ELBO and sample size

Statement

Let $m \gg 0$, $\lambda > 0$, $\frac{m}{\lambda} \in \mathbb{N}$, $\frac{m}{\lambda} \gg 0$. Then optimization

$$\mathsf{E}_q \log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) - \lambda \mathsf{D}_{\mathsf{KL}}\big(q(\boldsymbol{w})||p(\boldsymbol{w}|\boldsymbol{h})\big)$$

is equivalent to optimization of ELBO for a random subsample $\hat{\boldsymbol{y}}, \hat{\boldsymbol{X}}$ with size $\frac{m}{\lambda}$.

See also, [β-VAE, Fixing Broken ELBO, Talk from Olga Grebenkova, 2021].

Local reparametrization

How to calculate $\mathrm{E}_q \log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w})$?

- Graves, 2011: 1 sample per iteration. Use the following properties:

$$w \sim \mathcal{N}(\mu, \sigma^2) \to w \sim \varepsilon\sigma + \mu, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

  - Poor expectation approximation
- Naive solution: 1 sample per element in batch
  - BackProp will be very slow

Local reparametrization, Kingma et al., 2015

Let $y = \text{ReLU}(\boldsymbol{X}\boldsymbol{W})$ and parameter matrix $\boldsymbol{W}$ be distributed normally: $w_{i,j} \sim \mathcal{N}(\mu_{i,j}, \sigma^2_{i,j})$.
Then $\boldsymbol{X}\boldsymbol{W}$ is a Gaussian matrix:

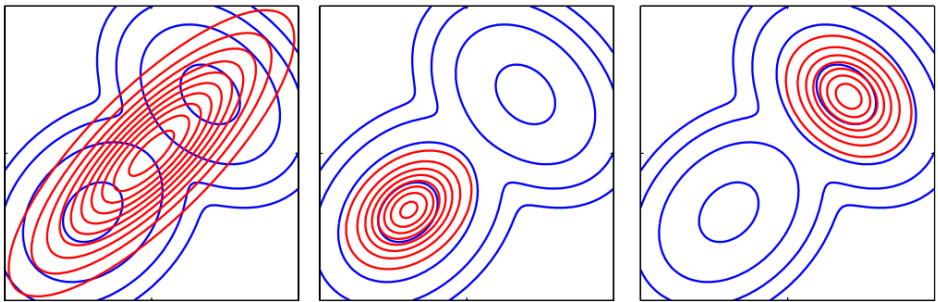
$$\boldsymbol{G} = \boldsymbol{X}\boldsymbol{W}, \quad G_{i,j} \sim \mathcal{N}(\sum_k x_{i,k}\mu_{k,j}, \sum_k x^2_{i,k}\sigma^2_{k,j}).$$

Instead of sampling parameters, sample elements from $\boldsymbol{G}$ (units before non-linear activation).

Example

Batch size = 64, matrix $\boldsymbol{W}$ dim is 64×64.

- Graves: one sample, $64 \times 64 = 4096$ elements. Poor approximation.
- Naive solution: sample parameters 64 times, $64 \times 64 \times 64 = 262144$ elements. Better approximation (in theory).
- Local reparametrization: sample $\boldsymbol{G}$, $64 \times 64 = 4096$ elements. Better approximation.



Expectation propagation

Minka, 2001: represent prior and approximation distribution via multiplication of factors, where number of factors (usually) corresponds to the sample size:

$$p(\boldsymbol{w}, \mathfrak{D}) = \prod_i f_i(\boldsymbol{w}), \quad q(\boldsymbol{w}) = \prod_i \tilde{f}_i(\boldsymbol{w})$$

Main idea — minimize $KL(p(\boldsymbol{w}|\mathfrak{D})|q(\boldsymbol{w}))$.

- Select factor $\tilde{f}_i$;
- Remove it to obtain the cavity distribution:

$$q^{\setminus i}(\boldsymbol{w}) \propto \prod_{j \neq i} \tilde{f}_j(\boldsymbol{w});$$

- Insert the exact factor:

$$\hat{p}_i(\boldsymbol{w}) \propto f_i(\boldsymbol{w}) q^{\setminus i}(\boldsymbol{w});$$

- Match moments of $\hat{p}_i$ and q;
- Repeat until convergence.

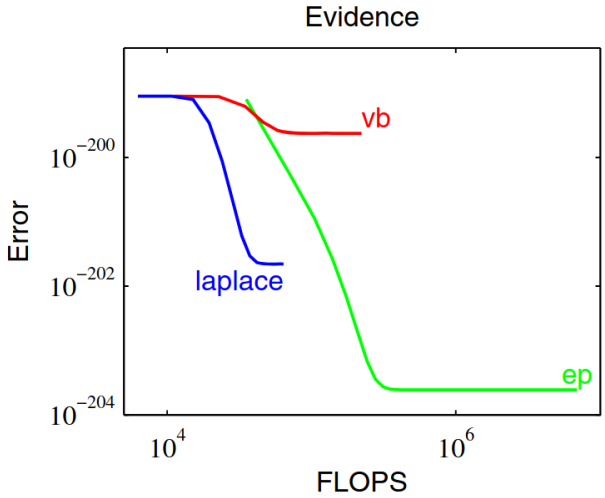
Expectation propagation: pros and cons

Cons:

- Assumption about posterior distribution (rather slight)
- Original version works only for q from exponential distribution
- No convergence guarantee
- Hard to decompose correlated objects
- Tricky to implement

Pros:

- Minimizes KL, not its lower bound
  - (but in a greedy way)



Plot for the 2-component Gaussian mixture.

Probabilistic backpropagation

Combination of Expectation propagation and backpropagation.[1]

Probabilistic model

Given data $\mathcal{D} = \{x_n, y_n\}_{n=1}^N$, made up of D-dimensional feature vectors and corresponding scalar target variables, we assume that $y_n = f(x_n; W) + \varepsilon_n$, where $f(;\mathbf{W})$ is the output of a multi-layer neural network with weights given by $\mathbf{W}$ and $\varepsilon_n \sim \mathcal{N}(0, \gamma^{-1})$.
Prior distributions:

$$p(W|\lambda) = \prod_{\mathbf{w}\in\mathbf{W}} \mathcal{N}(w|0, \lambda^{-1}),$$

$$p(\lambda) = \Gamma(\lambda|\alpha_0^\lambda, \beta_0^\lambda),$$

$$p(\gamma) = \Gamma(\gamma|\alpha_0^\gamma, \beta_0^\gamma).$$

[1] See talk of Polina Barabanshchikova, 2022

Theory

Likelihood for the weights $\mathbf{W}$ and the noise precision γ is

$$p(\mathbf{y}|\mathbf{W}, \mathbf{X}, \gamma) = \prod_{n=1}^{N} \mathcal{N}(y_n|f(x_n; \mathbf{W}), \gamma^{-1}).$$

The posterior distribution for $\mathbf{W}$, γ, λ

$$p(\mathbf{W}, \gamma, \lambda|\mathcal{D}) = \frac{p(\mathbf{y}|\mathbf{W}, \mathbf{X}, \gamma)p(\mathbf{W}|\lambda)p(\lambda)p(\gamma)}{p(\mathbf{y}|\mathbf{X})}.$$

Probabilistic backpropagation (PBP) approximates the exact posterior with a factored distribution given by

$$q(\mathbf{W}, \gamma, \lambda) = \prod_{\mathbf{w} \in \mathbf{W}} \mathcal{N}(\mathbf{w}|m_w, v_w) \times \Gamma(\gamma|\alpha^\gamma, \beta^\gamma)\Gamma(\lambda|\alpha^\lambda, \beta^\lambda).$$

Method description

Stages of PBP

1. In the first phase, the input data is propagated forward through the network. PBP sequentially approximates the marginal posterior distributions of each weight with a collection of one-dimensional Gaussians that match their marginal means and variances (**we could use local reparameteriation if there is no activation**). At the end of this phase, PBP computes the logarithm of the marginal probability of the target variable.

Method description

Stages of PBP

1. In the first phase, the input data is propagated forward through the network. PBP sequentially approximates the marginal posterior distributions of each weight with a collection of one-dimensional Gaussians that match their marginal means and variances (**we could use local reparameteriation if there is no activation**). At the end of this phase, PBP computes the logarithm of the marginal probability of the target variable.

2. In the second phase, the gradients of this quantity with respect to the means and variances of the approximate Gaussian posterior are propagated back. These derivatives are used to update the means and variances of the posterior approximation.

Method description

Update rule

Let $f(w)$ encode an arbitrary likelihood function for the single weight w and let our current beliefs regarding the scalar w be captured by a distribution $q(w)$. After seeing the data, our beliefs about w are updated according to Bayes' rule:

$$s(w) = Z^{-1} f(w) q(w),$$

where Z is the normalization constant.
We approximate this posterior with a distribution q^{new} that has the same form as q. The parameters of q^{new} are chosen to minimize the KL divergence between s and q^{new}.

Method description

Update rule (Example)

Assume that $q(w) = \mathcal{N}(w|m, v)$. In this case, the parameters of the new Gaussian beliefs $q^{new}(w) = \mathcal{N}(w|m^{new}, v^{new})$ that minimize the KL divergence between s and q^{new} can be obtained by

$$m^{new} = m + v\frac{\partial \log Z}{\partial m},$$

$$v^{new} = v - v^2\left[\left(\frac{\partial \log Z}{\partial m}\right)^2 - 2\frac{\partial \log Z}{\partial v}\right].$$

Method description

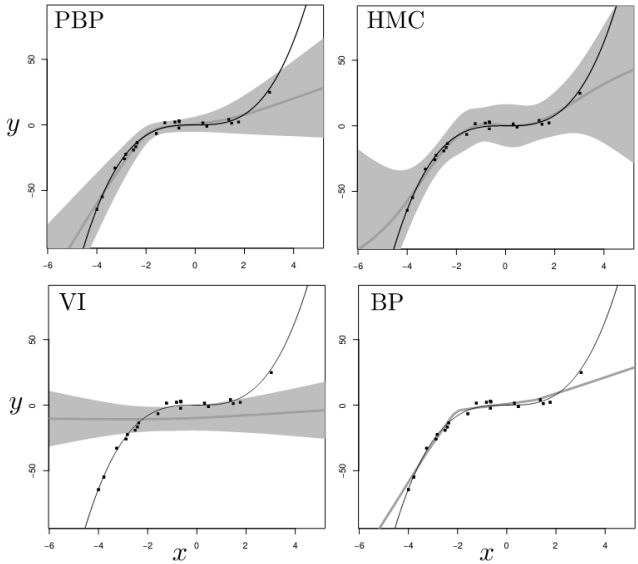
Update rule (Example)

Assume that $q(w) = \mathcal{N}(w|m, v)$. In this case, the parameters of the new Gaussian beliefs $q^{new}(w) = \mathcal{N}(w|m^{new}, v^{new})$ that minimize the KL divergence between s and q^{new} can be obtained by

$$m^{new} = m + v\frac{\partial \log Z}{\partial m},$$

$$v^{new} = v - v^2\left[\left(\frac{\partial \log Z}{\partial m}\right)^2 - 2\frac{\partial \log Z}{\partial v}\right].$$

Remark: Z is approximated during forward pass. Then its derivative is used to update the parameters of marginal distributions.


PBP
HMC
VI
BP
y
x
50
0
−50
−6
−4
−2
0
2
4

Reference

- Bishop C. M. Pattern recognition //Machine learning. – 2006. – Т. 128. – №. 9.
- MacKay D. J. C., Mac Kay D. J. C. Information theory, inference and learning algorithms. – Cambridge university press, 2003.
- Бахтеев О. Ю., Стрижов В. В. Выбор моделей глубокого обучения субоптимальной сложности //Автоматика и телемеханика. – 2018. – №. 8. – С. 129-147.
- Graves A. Practical variational inference for neural networks //Advances in neural information processing systems. – 2011. – Т. 24.
- Louizos C., Ullrich K., Welling M. Bayesian compression for deep learning //arXiv preprint arXiv:1705.08665. – 2017.
- Kingma D. P., Salimans T., Welling M. Variational dropout and the local reparameterization trick //Advances in neural information processing systems. – 2015. – Т. 28. – С. 2575-2583.
- Higgins I. et al. beta-vae: Learning basic visual concepts with a constrained variational framework. – 2016.
- Alemi A. et al. Fixing a broken ELBO //International Conference on Machine Learning. – PMLR, 2018. – С. 159-168.
- Minka T. P. Expectation propagation for approximate Bayesian inference //arXiv preprint arXiv:1301.2294. – 2013.
- Hernández-Lobato J. M., Adams R. Probabilistic backpropagation for scalable learning of bayesian neural networks //International conference on machine learning. – PMLR, 2015. – С. 1861-1869.

Organizational issues

Technical Meeting 2

- When: next week, at the regular meeting time.
- Duration: 15 minutes per team.
- Format: each team presents the current PoC implementation.
  - Show how the current code fits into the planned structure of your library.
  - Explain how the algorithms will be integrated into the library.
- The draft blog post and technical report will be reviewed offline.
- All materials presented at the meeting must be available in the project GitHub repository.
  - For the blog post and technical report, a read-only Overleaf link is acceptable if you are writing the documents there.

Blog Post and Technical Report

Minimum to write:

- Abstract / motivation.
- A planned structure with all sections and a short description of each section.
- All planned figures, either as actual figures or as detailed descriptions (*"There will be a scatter plot with our benchmark of algorithms. The x axis corresponds to the accuracy, y axis corresponds to the model size"*).
- A planned figure or figure description for the headline / main visual.
- Blog post: the text must be in English.
- Technical report: the text must be in English unless you plan to submit it to a Russian-language journal. In this case, explicitly mention this during the meeting.
- This is the minimum requirement. The more complete the document is, the better.

Documentation

- The documentation must be deployable:
  - either it is already available online;
  - or there are clear instructions for deploying it in standalone mode.
  - By the end of the project, the documentation must be deployed online.
- The documentation structure must already be established.
  - Look at the documentation of your favorite libraries for examples.
- The documentation must be runnable:
  - online (e.g., GitHub Pages), or
  - locally on your PC.
- For the demo, show at least one automatically generated documentation page.
- Popular tools: MkDocs, Sphinx.

Unit Tests (not required yet)

- Cover as many functions and code paths as possible.
- Target minimum: 75% code coverage.
- Synthetic tests are welcome:
  - generate a dataset on the fly;
  - check that the implemented functions produce the expected results;
- popular engines: unittest (built-in into python) + coverage, pytest + pytest-cov.

Next Deadlines

- **Checkpoint:** November 3 or November 11.
- **Technical Meeting 3:** November 24.
  - All project components must be completed by this meeting.
- Reminder: comments on previous tech. meeting (the scores will be zero after tech. meeting 2)
- Reminder: do not forget to prepare the blog posts about your talks.